Prim Algorithm

Group 3

03/2026

Định nghĩa

Prim là gì?

Prim là thuật toán tìm cây khung nhỏ nhất (Minimum Spanning Tree - MST) trong một đồ thị vô hướng, liên thông, có trọng số.

- Bắt đầu từ một đỉnh bất kỳ
- Mở rộng dần cây khung bằng cách chọn cạnh nhỏ nhất
- Không tạo chu trình

Cây khung nhỏ nhất là gì?

- Cây khung là tập cạnh nối tất cả các đỉnh của đồ thị
- Có đúng $n - 1$ cạnh nếu đồ thị có n đỉnh
- Không có chu trình
- Tổng trọng số nhỏ nhất trong tất cả các cây khung

Prim xây dựng MST bằng cách mở rộng từ một đỉnh gốc

Ý tưởng của Prim

1. Chọn một đỉnh bắt đầu
2. Luôn giữ tập đỉnh đã thuộc MST
3. Từ tập này, chọn cạnh nhỏ nhất nối ra ngoài
4. Thêm đỉnh mới vào MST
5. Lặp lại cho đến khi có đủ n đỉnh

Prim mở rộng cây từng bước từ bên trong ra ngoài

Bản chất

- Thuật toán Greedy (tham lam)
- Mỗi bước chọn cạnh nhỏ nhất hợp lệ
- Cạnh được chọn phải nối từ cây hiện tại ra một đỉnh chưa thăm

Mỗi lần thêm một đỉnh mới vào cây khung

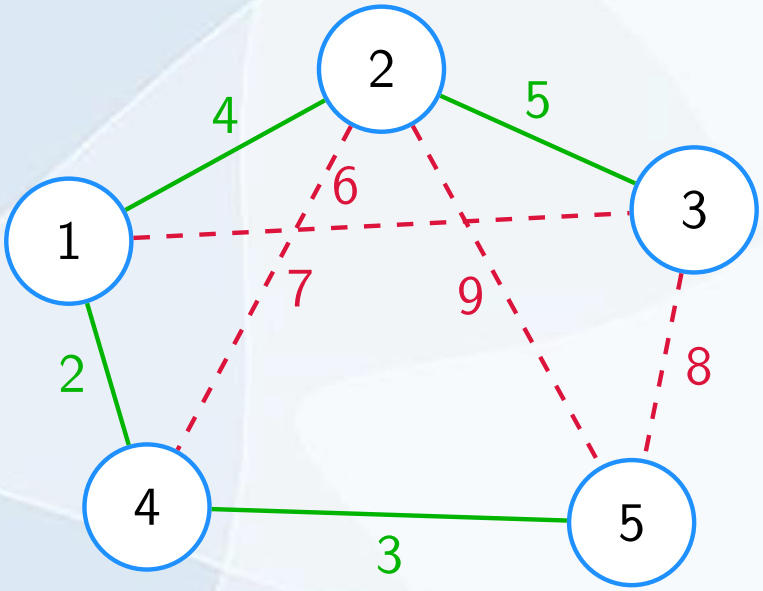
Cấu trúc dữ liệu cần dùng

- Danh sách kề của đồ thị
- Priority queue / min-heap
- Mảng `visited` để đánh dấu đỉnh đã thuộc MST
- Mảng `parent` nếu muốn truy vết cạnh của cây khung

Vai trò của heap

Heap giúp ta nhanh chóng lấy ra cạnh có trọng số nhỏ nhất đang nối từ cây hiện tại ra ngoài.

Minh họa trực quan



- Bắt đầu từ đỉnh 1
- Mỗi lần chọn cạnh rẻ nhất nối ra ngoài
- Cạnh màu xanh là cạnh được chọn
- Cạnh nét đứt là cạnh chưa chọn

Quy trình chi tiết

1. Chọn một đỉnh xuất phát
2. Đưa các cạnh kề của đỉnh đó vào heap
3. Lấy cạnh nhỏ nhất từ heap
4. Nếu đỉnh đầu kia chưa thăm:
   - thêm cạnh vào MST
   - đánh dấu đỉnh mới
   - đưa các cạnh kề của nó vào heap
5. Nếu đỉnh đó đã thăm thì bỏ qua
6. Lặp lại đến khi có đủ $n - 1$ cạnh

Ví dụ từng bước

Bước	Cạnh xét	Quyết định	Lý do
1	1–4 (2)	Chọn	Nhỏ nhất từ đỉnh 1
2	4–5 (3)	Chọn	Kết nối thêm đỉnh mới
3	1–2 (4)	Chọn	Đỉnh 2 chưa thăm
4	2–3 (5)	Chọn	Đỉnh 3 chưa thăm
5	1–3 (6)	Bỏ	Đỉnh 3 đã thuộc MST
6	2–4 (7)	Bỏ	Cả hai đầu đã trong MST

Sau 4 cạnh được chọn, ta có MST cho 5 đỉnh

Tính chất quan trọng

- Greedy(Tham lam): luôn chọn cạnh tốt nhất ở thời điểm hiện tại
- An toàn: cạnh được chọn không làm mất tính tối ưu
- Không tạo chu trình: vì chỉ nối ra đỉnh chưa thăm
- Phù hợp với đồ thị có nhiều cạnh kề từ một đỉnh

Vì sao Prim đúng?

- Mỗi bước chỉ chọn cạnh nhỏ nhất nối từ cây hiện tại ra ngoài
- Cạnh đó luôn là lựa chọn an toàn theo tính chất cắt của MST
- Lặp lại đủ $n - 1$ lần sẽ tạo ra một cây khung nhỏ nhất

Cốt lõi: mở rộng cây bằng cạnh nhẹ nhất có thể

Code C++ (1/2)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100005;
vector<pair<int,long long>> adj[MAXN];
bool visited[MAXN];
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < m; i++){
        int u, v;
        long long w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    priority_queue<pair<long long,int>, vector<pair<long long,
        int>>, greater<pair<long long,int>>> pq;
```

Code C++ (2/2)

```
    int start = 1;
    visited[start] = true;
    for(auto [v, w] : adj[start]) pq.push({w, v});
    long long mst_cost = 0;
    int cnt = 1;
    vector<pair<int,int>> edges;
    while(!pq.empty() && cnt < n){
        auto [w, u] = pq.top();
        pq.pop();
        if(visited[u]) continue;
        visited[u] = true;
        mst_cost += w;
        cnt++;
        for(auto [v, w2] : adj[u])
            if(!visited[v]) pq.push({w2, v});
    }
    cout << mst_cost << '\n';
    return 0;
}
```

Code có truy vết cạnh

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100005;
vector<pair<int,long long>> adj[MAXN];
bool visited[MAXN];
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < m; i++){
        int u, v;
        long long w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    priority_queue<tuple<long long,int,int>, vector<tuple<long
        long,int,int>>, greater<tuple<long long,int,int>>> pq;
```

Code có truy vết cạnh (tiếp)

```
    int start = 1;    visited[start] = true;
    for(auto [v, w] : adj[start]) pq.push({w, start, v});
    long long mst_cost = 0;
    vector<tuple<int,int,long long>> mst_edges;
    while(!pq.empty() && (int)mst_edges.size() < n - 1){
        auto [w, u, v] = pq.top();
        pq.pop();
        if(visited[v]) continue;
        visited[v] = true;
        mst_cost += w;
        mst_edges.push_back({u, v, w});
        for(auto [to, w2] : adj[v])
            if(!visited[to]) pq.push({w2, v, to});
    }
    cout << mst_cost << '\n';
    for(auto [u, v, w] : mst_edges)
        cout << u << ' ' << v << ' ' << w << '\n';
    return 0;
}
```

Độ phức tạp

- Với heap:
  - Mỗi lần push / pop: $O(\log m)$
  - Duyệt cạnh qua danh sách kề
- Tổng độ phức tạp thường dùng:

$$O(m \log m) \text{ hoặc } O(m \log n)$$

- Tùy cách cài đặt và biểu diễn heap

Lỗi thường gặp

- Quên đánh dấu `visited`
- Đẩy lại cạnh của đỉnh đã thăm gây dư thừa
- Không kiểm tra đồ thị liên thông
- Không dùng `long long` cho trọng số lớn
- Nhầm Prim với Dijkstra

So sánh với Kruskal

Tiêu chí	Prim	Kruskal
Cách làm	Mở rộng từ một đỉnh	Chọn cạnh nhỏ nhất
Cấu trúc dữ liệu	Heap, visited	DSU, sort cạnh
Phù hợp	Đồ thị dày	Đồ thị thưa
Ý tưởng chính	Mở rộng cây	Gộp các thành phần

Ứng dụng thực tế

- Thiết kế mạng điện, mạng đường, mạng máy tính
- Tối ưu chi phí kết nối
- Bài toán quy hoạch hạ tầng
- Xuất hiện nhiều trong lập trình thi đấu

Kết luận

- Prim là thuật toán kinh điển để tìm MST
- Ý tưởng dễ nhớ: luôn mở rộng từ cây hiện tại bằng cạnh nhỏ nhất
- Dùng heap giúp cài đặt hiệu quả
- Hiểu tốt Prim sẽ hỗ trợ rất nhiều cho các bài toán đồ thị

Prim là một trong những thuật toán greedy nền tảng nhất trong đồ thị